

웹의 세계 5장

웹 직군과 개발 생태계

누가 무엇을 만드는가, 어떤 도구를 쓰는가, 1학년이 지금 할 일

한신대학교 AI·SW계열 · 웹프로그래밍
학습사이트 hs-web.dreamitbiz.com

학습 목표

- ① 웹을 만드는 직군과 각각의 역할을 설명할 수 있다.
- ② 프론트엔드 생태계의 주요 도구와 그 존재 이유를 설명할 수 있다.
- ③ 프레임워크가 무엇을 해결하려고 나왔는지 설명할 수 있다.
- ④ 학습 순서를 스스로 계획할 수 있다.
- ⑤ 1학년 2학기에 지금 무엇을 하면 좋은지 판단할 수 있다.

웹의 세계 5장

핵심 키워드

프론트엔드

백엔드

풀스택

UI/UX

DevOps

React

Vue

TypeScript

Git

학습 로드맵

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

웹의 세계 5장 · 1

웹을 만드는 사람들

웹을 만드는 사람들 (1/2)

직군	하는 일	주로 쓰는 것
프론트엔드	사용자가 보고 만지는 화면	HTML·CSS·JS, React·Vue
백엔드	자료 저장·조회, 인증, 권한, 계산	Node.js·Python·Java, DB
풀스택	양쪽 모두	위 전부
UI/UX 디자이너	화면 설계와 사용 흐름	Figma

웹을 만드는 사람들 (2/2)

직군	하는 일	주로 쓰는 것
기획자(PM/PO)	무엇을 만들지 정하고 우선순위를 조율	문서, 데이터
QA	품질 검증, 버그 발견	테스트 도구
DevOps/SRE	배포 자동화, 서버 운영, 모니터링	Docker, AWS, CI/CD
데이터 엔지니어	데이터 수집·가공 파이프라인	Python, SQL

웹을 만드는 사람들

- 1 혼자 만드는 서비스는 거의 없습니다.
- 2 웹 개발은 협업이고, 그래서 코드를 남이 읽을 수 있게 쓰는 것과 소통이 실력의 일부입니다.
- 3 팀 프로젝트(14~15주차)를 하는 이유이기도 합니다.

1.1 프론트엔드가 실제로 하는 일 (1/2)

1 "화면 만드는 사람"이라고 하면 단순해 보이지만, 실제 업무는 훨씬 넓습니다.

2 **화면 구현**
디자이너의 시안을 실제 동작하는 화면으로.

3 **상태 관리**
로그인 여부, 장바구니, 필터 조건 같은 것을 일관되게 다루기.

4 **서버 연동**
API를 호출하고 로딩·오류·빈 상태를 모두 처리.

1.1 프론트엔드가 실제로 하는 일 (2/2)

1 성능

느린 페이지를 빠르게. 이미지·번들 크기·렌더링 최적화.

2 접근성·호환성

모든 사용자, 모든 브라우저에서 동작하게.

3 협업

디자이너와 시안을 조율하고, 백엔드와 API 형식을 합의.

1.1 프론트엔드가 실제로 하는 일

- 1 "디자인 감각이 없어서 프론트는 못 하겠다"고 생각할 필요는 없습니다.
- 2 디자인은 디자이너가 합니다.
- 3 프론트엔드는 그것을 정확하게, 빠르게, 모든 환경에서 동작하게 구현하는 일이고, 그것은 논리와 꼼꼼함의 영역입니다.

웹의 세계 5장 · 2

도구 생태계 — 왜 이렇게 많은가

프론트엔드는 도구가 많기로 악명이 높습니다.

도구 생태계 — 왜 이렇게 많은가

도구	없을 때의 문제	해결
Git	"과제_최종_진짜최종.html"	버전 관리
npm	라이브러리를 손으로 내려받아 관리	패키지 관리
Vite / webpack	파일이 수백 개, 로딩이 느림	번들링·최적화
Sass / PostCSS	CSS에 변수·중첩이 없던 시절	CSS 확장 (지금은 CSS 자체가 많이 발전)
TypeScript	undefined is not a function 을 실행해 봐야 앎	타입 검사 — 실행 전에 잡음
ESLint / Prettier	팀원마다 코드 스타일이 다름	자동 검사·정렬
React / Vue	DOM을 손으로 고치다 관리 불능	상태 기반 자동 렌더링

하지만 각각이 해결하려던 문제를 알면 훨씬 덜 무섭습니다.

지금 다 알 필요는 없습니다

● 정리

- 1 모든 도구는 문제가 생긴 뒤에 배우는 것이 빠릅니다.**
문제를 겪어 보지 않고 배우면 왜 쓰는지 모른 채 외우게 됩니다.
- 2 이 과목에서는 Git과 GitHub Pages만**
씁니다(15주차). 그것으로 충분합니다.
- 3 도구는 바뀌지만 HTML·CSS·JS는 안 바뀝니다.**
바닥에 시간을 쓰세요.

웹의 세계 5장 · 3

프레임워크 — 무엇을 해결하는가

11주차에 손으로 만든 "상태를 바꾸고 render()를 부른다" 구조를 기억하시나요?

프레임워크 — 무엇을 해결하는가

	React	Vue	Svelte
만든 곳	Meta(페이스북)	개인(에반 유)에서 시작	개인에서 시작
특징	가장 큰 생태계, 채용 수요 1위	배우기 쉽고 문서가 친절	빌드 시점에 최적화, 가장 가벼움
한국 실무	가장 많이 씀	두 번째로 많이 씀	점점 늘어나는 중
학습 난이도	중간	쉬움	쉬움

프레임워크는 그 `render()` 호출을 자동으로 해 줍니다.

프레임워크 — 무엇을 해결하는가

- 1 어느 것을 배워도 괜찮습니다.
- 2 하나를 제대로 익히면 다른 것으로 옮기는 데 며칠이면 됩니다.
- 3 개념(컴포넌트·상태·props)이 같기 때문입니다.
- 4 다만 HTML·CSS·JS를 모른 채 프레임워크부터 배우면 반드시 막힙니다.

3.1 프레임워크가 필요 없는 경우

- 1 정적인 소개 사이트·포트폴리오**
HTML·CSS로 충분하고, 오히려 더 빠릅니다.
- 2 간단한 랜딩 페이지**
프레임워크를 없으면 파일 크기만 커집니다.
- 3 이 과목의 팀 프로젝트**
순수 HTML·CSS·JS로 만드는 것이 목표입니다.

3.1 프레임워크가 필요 없는 경우

- 1 "React를 쓰는 것이 무조건 좋다"는 것은 오해입니다.
- 2 문제 크기에 맞는 도구를 고르는 것이 실력입니다.

웹의 세계 5장 · 4

학습 로드맵

이 순서를 건너뛰면 반드시 되돌아오게 됩니다

```
1  [ 지금 여기 ]
2  HTML · CSS · JavaScript
3      ↓
4  Git · GitHub · 배포          ← 15주차
5      ↓
6  직접 만들어 보기 (아주 중요) ← 방학
7      ↓
8  React 또는 Vue 하나
9      ↓
10 TypeScript
11      ↓
12  ┌───────────┴───────────┐
13  프론트 심화   백엔드
14  (성능 · 테스트) (Node.js · DB)
```

4.2 단계마다 무엇을 해야 하는가

단계	"됐다"의 기준
HTML·CSS	디자인 시안을 보고 비슷하게 만들 수 있다. 반응형이 된다
JavaScript	값을 다루고, 화면을 바꾸고, API를 불러올 수 있다
Git·배포	내 사이트를 인터넷에 공개하고 계속 갱신할 수 있다
프레임워크	컴포넌트로 나누고 상태를 관리할 수 있다
TypeScript	타입 오류의 의미를 읽고 고칠 수 있다

가장 중요한 조언

● 정리

- 1 강의를 듣는 것보다 만드는 것이 빠릅니다.**
강의 열 개보다 완성한 프로젝트 하나가 훨씬 많이 남습니다.
- 2 작게, 끝까지.**
거창한 계획은 대부분 중간에 멈춥니다. 페이지 3개짜리를 끝내는 것이 낫습니다.
- 3 남의 코드를 읽으세요.**
마음에 드는 사이트를 F12로 뜯어보는 것이 가장 좋은 공부입니다.
- 4 막히면 30분 안에 물어보세요.**
혼자 오래 붙드는 것은 인내가 아니라 낭비입니다.

웹의 세계 5장 · 5

1학년 2학기에 지금 할 일

이번 학기 안에

● 정리

- 1 GitHub 계정을 만들고 커밋을 쌓기 시작하세요.**
사용자 이름은 사용자.github.io 라는 평생 주소가 되니 신중히 정하세요.
- 2 이 과목의 프로젝트를 끝까지 완성**
하세요. 실제 주소가 있는 결과물 하나가 생깁니다.
- 3 개발자도구를 매일 열어 보세요.**
이것이 습관이 되면 나중에 훨씬 편합니다.

방학에 하면 좋은 것

● 정리

- 1 이번 학기 프로젝트를 확장**
하세요. 기능을 하나씩 붙여 보는 것이 새것을 시작하는 것보다 훨씬 많이 배웁니다.
- 2 만들고 싶은 것을 하나 정해 끝까지 만드세요.**
남의 튜토리얼을 따라 하는 것과 내 것을 만드는 것은 다릅니다.
- 3 React나 Vue 중 하나를 맛보세요.**
다만 HTML·CSS·JS가 편해진 뒤에.

5.1 조급해하지 않아도 되는 이유 (1/2)

- 1 2학년, 3학년에 배울 것이 많이 남아 있습니다.
- 2 1학년 2학기에 웹을 만들어 인터넷에 공개해 본 것만으로도 충분히 앞서 있는 것입니다.
- 3 지금 필요한 것은 진도가 아니라 "만드는 것이 재미있다"는 경험입니다.

5.1 조금해하지 않아도 되는 이유 (2/2)

- 1 이력서에 쓸 것은 "React를 안다"가 아니라 "무엇을 만들어 봤다" 입니다.
- 2 채용에서 가장 먼저 보는 것도 GitHub와 만든 것입니다.
- 3 그러니 학기 프로젝트를 진짜로 완성하세요 — 그것이 첫 칸입니다.

웹의 세계 5장 · 6

참고할 만한 곳

참고할 만한 곳

이름	무엇	언제
MDN Web Docs	웹 기술 공식 문서 (한국어 상당수 번역)	문법·API가 헛갈릴 때 — 가장 정확
caniuse.com	브라우저 지원 현황	새 기능을 써도 되나 확인할 때
MDN Learn	단계별 학습 자료	체계적으로 복습하고 싶을 때
web.dev	구글의 웹 품질 가이드	성능·접근성을 깊이 볼 때
Stack Overflow	질문·답변	구체적인 오류를 검색할 때

참고할 만한 곳

- 1 블로그 글은 낡았을 수 있습니다.
- 2 2015년 글의 방법이 지금은 권장되지 않는 경우가 흔합니다.
- 3 헛갈리면 MDN을 기준으로 삼으세요.
- 4 AI에게 물을 때도 "MDN 기준으로 알려 줘"라고 덧붙이면 답이 정확해집니다.

웹의 세계 5장 마무리

5장 마무리

웹 직군과 개발 생태계 — 본문과 도해는 학습사이트에서 다시 읽습니다

✓ 학습사이트 장 페이지 끝의 「이해했어요 · 어려워요」 표시

✓ 장 끝의 확인 퀴즈로 스스로 점검

✓ 함께 볼 주차: 13주차 · 14주차

✓ 막히면 「질문있어요」 — 질문은 반 전체가 같은 자리에서 봅니다